Grader Output

Lab3-TidyModels

100.00 / 100.00 points earned
2 / 2 autograded cells passed

Graded Cells

Cell 13 (cell-737ad03a2bff49ee)

Passed | 50.00 / 50.00 points
View feedback

Cell 15 (cell-514f8c74a301fae5)

Passed | 50.00 / 50.00 points
View feedback

Linear Regression using TidyModels

In this lab exercise we would be going through

- simple linear regression
- multiple linear regression
- transformations to predictors (using `parsnip`)

In [1]:

```
suppressPackageStartupMessages(library(tidymodels))
suppressPackageStartupMessages(library(ISLR))
suppressPackageStartupMessages(library(MASS))
```

In [2]:

```
head(petrol)
```

A data.frame: 6 × 6

	No	SG	VP	V10	EP	Y
	<fct>	<dbl>	<dbl>	<int>	<int>	<dbl>
1	A	50.8	8.6	190	205	12.2
2	A	50.8	8.6	190	275	22.3
3	A	50.8	8.6	190	345	34.7
4	A	50.8	8.6	190	407	45.7
5	B	40.8	3.5	210	218	8.0
6	B	40.8	3.5	210	273	13.1

Simple Linear Regression

We are using `Boston` data set - contains various statistics for 506 neighborhoods in Boston

Agenda: Build a simple linear regression model that related the median value of owner-occupied homes (`medv`) as the response with a variable indicating the percentage of the population that belongs to a lower status (`lstat`) as the predictor.

In the below step, we create a parsnip specification for a linear regression model

In [3]:

```
lm_spec <- linear_reg() %>%
  set_mode("regression") %>%
  set_engine("lm")
```

In [4]:

```
lm_spec
```

```
Linear Regression Model Specification (regression)

Computational engine: lm
```

In [5]:

```
head(Boston)
```

A data.frame: 6 × 14

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	l
	<dbl>	<dbl>	<dbl>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<dbl>	<dbl>	<
1	0.00632	18	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	3
2	0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	3
3	0.02729	0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	3
4	0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	3
5	0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	3
6	0.02985	0	2.18	0	0.458	6.430	58.7	6.0622	3	222	18.7	3

Once we have the specification we can fit it by supplying a formula expression and the data we want to fit the model on.

The formula is written on the form `y ~ x` where `y` is the name of the response and `x` is the name of the predictors. The names used in the formula should match the names of the variables in the data set passed to data.

In [6]:

```
lm_fit <- lm_spec %>% fit(medv ~ lstat, data = Boston)
lm_fit
```

```
parsnip model object

Fit time:  4ms

Call:
stats::lm(formula = medv ~ lstat, data = data)

Coefficients:
(Intercept)        lstat
      34.55        -0.95
```

The result of this fit is a parsnip model object. This object contains the underlying fit as well as some parsnip-specific information. If we want to look at the underlying fit object we can access it with `lm_fit$fit` or with

In [7]:

```
lm_fit %>%
  pluck("fit")
```

```
Call:
stats::lm(formula = medv ~ lstat, data = data)

Coefficients:
(Intercept)        lstat
      34.55        -0.95
```

In [8]:

```
lm_fit %>%
  pluck("fit") %>%
  summary()
```

```
Call:
stats::lm(formula = medv ~ lstat, data = data)

Residuals:
    Min      1Q  Median      3Q     Max
-15.168  -3.990  -1.318   2.034  24.500

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 34.55384    0.56263   61.41   <2e-16 ***
lstat       -0.95005    0.03873  -24.53   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 6.216 on 504 degrees of freedom
Multiple R-squared:  0.5441,    Adjusted R-squared:  0.5432
F-statistic: 601.6 on 1 and 504 DF,  p-value: < 2.2e-16
```

`tidy()` function returns the parameter estimates of a lm object

In [9]:

```
tidy(lm_fit)
```

A tibble: 2 × 5

term	estimate	std.error	statistic	p.value
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
(Intercept)	34.5538409	0.56262735	61.41515	3.743081e-236
lstat	-0.9500494	0.03873342	-24.52790	5.081103e-88

`glance()` can be used to extract the model statistics

In [10]:

```
glance(lm_fit)
```

A tibble: 1 × 12

r.squared	adj.r.squared	sigma	statistic	p.value	df	logLik	AIC	
<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	
0.5441463	0.5432418	6.21576	601.6179	5.081103e-88	1	-1641.487	3288.975	3:

If we like the model fit then we can generate the predictions using the `predict()` function

In [11]:

```
predict(lm_fit, new_data = Boston)
```

A tibble: 506 × 1

.pred
<dbl>
29.822595
25.870390
30.725142
31.760696
29.490078
29.604084
22.744727
16.360396
6.118864
18.307997
15.125332
21.946686
19.628566
26.706433
24.806335
26.506923
28.302516
20.616617
23.447764
23.837284
14.583803
21.414658
16.768917
15.666860
19.068036
18.868526
20.483610
18.136988
22.393209
23.172250
⋮
16.806919
10.888111
17.424451
22.098694
24.350311

.pred
<dbl>
27.200459
27.893995
24.654327
21.880183
24.502319
20.322102
23.675776
17.395950
11.781158
6.356376
17.386449
21.870682
23.143748
21.642670
17.832972
14.469798
21.158145
22.279203
20.208096
20.939634
25.366864
25.927393
29.195563
28.397521
27.067452

Excercise

Agenda: Build a simple linear regression model that relates `medv` as response to `age` as the predictor

In [12]:

Student's answer (Top)

```
# your code here
lm.model <- function() {
    return (lm_spec %>% fit(medv ~ age, data = Boston))
}
```

In [13]:

Grade cell: `cell-737ad03a2bff49ee` Score: 50.0 / 50.0 (Top)

```
#hidden test cases

Hidden Tests Redacted
```

In [14]:

Student's answer (Top)

```
# your code here
lm.predict <- function() {
    #tidy(lm_fit)
    return(predict(lm.model(), new_data = Boston))
}
```

In [15]:

Grade cell: `cell-514f8c74a301fae5` Score: 50.0 / 50.0 (Top)

```
#hidden test cases

Hidden Tests Redacted
```

Multiple Linear Regression

The multiple linear regression model can be fit in much the same way as the simple linear regression model. The only difference is how we specify the predictors. We are using the same formula expression `y ~ x` , but we can specify multiple values by separating them with `+s`

In [16]:

```
lm_fit2 <- lm_spec %>%
  fit(medv ~ lstat + age, data = Boston)

lm_fit2
```

```
parsnip model object

Fit time:  1ms

Call:
stats::lm(formula = medv ~ lstat + age, data = data)

Coefficients:
(Intercept)        lstat          age
   33.22276     -1.03207      0.03454
```

In [17]:

```
tidy(lm_fit2)
predict(lm_fit2, new_data = Boston)
```

A tibble: 3 × 5

term	estimate	std.error	statistic	p.value
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
(Intercept)	33.22276053	0.73084711	45.457881	2.943785e-180
lstat	-1.03206856	0.04819073	-21.416330	8.419554e-73
age	0.03454434	0.01222547	2.825605	4.906776e-03

A tibble: 506 × 1

.pred
<dbl>
30.335350
26.515202
31.174183
31.770610
29.594138
29.873436
22.694801
16.778358
5.787382
18.541747
15.374490
22.390936
18.356193
26.832714
25.552734
26.432895
27.443898
20.904587
22.422202
23.981859
14.917479
22.030607
17.070153
16.159671
19.650665
19.143758
21.057179
18.456153
23.273268
23.874300
⋮
17.177070
10.875097
17.955002
22.732243
24.373363

.pred
<dbl>
27.821921
28.647874
23.860743
20.902374
24.096326
20.519012
23.243334
17.785862
11.879786
5.976311
17.986262
22.329097
22.693011
20.668538
16.053231
13.923113
21.109424
22.144180
20.177534
21.186402
25.629671
26.501129
30.545429
29.619766
27.881243

A shortcut when using formulas is to use the form `y ~ .` which means; set `y` as the response and set the remaining variables as predictors

In [18]:

```
lm_fit3 <- lm_spec %>%
  fit(medv ~ ., data = Boston)

lm_fit3
```

```
parsnip model object

Fit time:  2ms

Call:
stats::lm(formula = medv ~ ., data = data)

Coefficients:
(Intercept)         crim           zn        indus         chas
nox
  3.646e+01   -1.080e-01    4.642e-02    2.056e-02    2.687e+00   -
1.777e+01
         rm          age          dis          rad          tax
ptratio
  3.810e+00    6.922e-04   -1.476e+00    3.060e-01   -1.233e-02   -
9.527e-01
      black        lstat
  9.312e-03   -5.248e-01
```

Interaction Terms

An interaction term is represented as the product of two or more independent variables/predictors

There are two ways on including an interaction term; `x:y` and `x * y`

- `x:y` will include the interaction between `x` and `y`
- `x * y` will include the interaction between `x` and `y`, `x` and `y`, i.e. it is short for `x:y + x + y`

In [19]:

```
lm_fit4 <- lm_spec %>%
  fit(medv ~ lstat * age, data = Boston)

lm_fit4
```

```
parsnip model object

Fit time:  2ms

Call:
stats::lm(formula = medv ~ lstat * age, data = data)

Coefficients:
(Intercept)        lstat          age    lstat:age
 36.0885359   -1.3921168   -0.0007209    0.0041560
```

note that the interaction term is named `lstat:age`.

Sometimes we want to perform transformations, and we want those transformations to be applied, as part of the model fit as a pre-processing step. We will use the recipes package for this task.

We use the `step_interact()` to specify the interaction term. Next, we create a workflow object to combine the linear regression model specification lm_spec with the pre-processing specification `rec_spec_interact` which can then be fitted much like a parsnip model specification.

In [20]:

```
rec_spec_interact <- recipe(medv ~ lstat + age, data = Boston) %>%
  step_interact(~ lstat:age)

lm_wf_interact <- workflow() %>%
  add_model(lm_spec) %>%
  add_recipe(rec_spec_interact)

lm_wf_interact %>% fit(Boston)
```

```
══ Workflow [trained] ══════════════════════════════════════════════════════
Preprocessor: Recipe
Model: linear_reg()

── Preprocessor ────────────────────────────────────────────────────────────
1 Recipe Step

• step_interact()

── Model ───────────────────────────────────────────────────────────────────

Call:
stats::lm(formula = ..y ~ ., data = data)

Coefficients:
(Intercept)        lstat          age  lstat_x_age
 36.0885359   -1.3921168   -0.0007209    0.0041560
```

Notice that since we specified the variables in the recipe we don't need to specify them when fitting the workflow object. Furthermore, take note of the name of the interaction term. `step_interact()` tries to avoid special characters in variables

Non-linear transformations of the predictors

Much like we could use recipes to create interaction terms between values are we able to apply transformations to individual variables as well. If you are familiar with the dplyr package then you know how to `mutate()` which works in much the same way using `step_mutate()`.

You would want to keep as much of the pre-processing inside recipes such that the transformation will be applied consistently to new data.

In [21]:

```
rec_spec_pow2 <- recipe(medv ~ lstat, data = Boston) %>%
  step_mutate(lstat2 = lstat ^ 2)

lm_wf_pow2 <- workflow() %>%
  add_model(lm_spec) %>%
  add_recipe(rec_spec_pow2)

lm_wf_pow2 %>% fit(Boston)
```

```
══ Workflow [trained] ══════════════════════════════════════════════════════
Preprocessor: Recipe
Model: linear_reg()

── Preprocessor ────────────────────────────────────────────────────────────
1 Recipe Step

• step_mutate()

── Model ───────────────────────────────────────────────────────────────────

Call:
stats::lm(formula = ..y ~ ., data = data)

Coefficients:
(Intercept)        lstat       lstat2
   42.86201     -2.33282      0.04355
```

Qualitative Predictors

We will now turn our attention to the `Carseats` data set. We will attempt to predict `Sales` of child car seats in 400 locations based on a number of predictors. One of these variables is `ShelveLoc` which is a qualitative predictor that indicates the quality of the shelving location.

`ShelveLoc` takes on three possible values

- Bad
- Medium
- Good

If you pass such a variable to `lm()` it will read it and generate dummy variables automatically using the following convention

In [22]:

```
Carseats %>%
  pull(ShelveLoc) %>%
  contrasts()
```

A matrix: 3 × 2 of type dbl

	Good	Medium
Bad	0	0
Good	1	0
Medium	0	1

So we have no problems including qualitative predictors when using `lm` as the engine.

In [23]:

```
lm_spec %>%
  fit(Sales ~ . + Income:Advertising + Price:Age, data = Carseats)
```

```
parsnip model object

Fit time:  3ms

Call:
stats::lm(formula = Sales ~ . + Income:Advertising + Price:Age,
    data = data)

Coefficients:
       (Intercept)           CompPrice              Income         A
dvertising
         6.5755654           0.0929371           0.0108940
0.0702462
        Population               Price       ShelveLocGood     Shelv
eLocMedium
         0.0001592          -0.1008064           4.8486762
1.9532620
               Age           Education            UrbanYes
USYes
        -0.0579466          -0.0208525           0.1401597
-0.1575571
Income:Advertising           Price:Age
         0.0007510           0.0001068
```

however, as with so many things, we can not always guarantee that the underlying engine knows how to deal with qualitative variables. recipes can be used to handle this as well. The `step_dummy()` will perform the same transformation of turning 1 qualitative with `C` levels into `C-1` indicator variables.

While this might seem unnecessary right now, some of the engines, later on, do not handle qualitative variables and this step would be necessary.

We are also using the `all_nominal_predictors()` selector to select all character and factor predictor variables. This allows us to select by type rather than having to type out the names.

In [24]:

```
rec_spec <- recipe(Sales ~ ., data = Carseats) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(~ Income:Advertising + Price:Age)

lm_wf <- workflow() %>%
  add_model(lm_spec) %>%
  add_recipe(rec_spec)

lm_wf %>% fit(Carseats)
```

```
══ Workflow [trained] ══════════════════════════════════════════════════════
Preprocessor: Recipe
Model: linear_reg()

── Preprocessor ────────────────────────────────────────────────────────────
2 Recipe Steps

• step_dummy()
• step_interact()

── Model ───────────────────────────────────────────────────────────────────

Call:
stats::lm(formula = ..y ~ ., data = data)

Coefficients:
         (Intercept)            CompPrice               Income
           6.5755654            0.0929371            0.0108940
         Advertising           Population                Price
           0.0702462            0.0001592           -0.1008064
                 Age            Education       ShelveLoc_Good
          -0.0579466           -0.0208525            4.8486762
    ShelveLoc_Medium            Urban_Yes               US_Yes
           1.9532620            0.1401597           -0.1575571
Income_x_Advertising          Price_x_Age
           0.0007510            0.0001068
```